INFORME TÉCNICO

PRUEBAS DE CAJA BLANCA Y CAJA NEGRA

Operación transaccional: asignación de turno · función store

Sistema de Control de Transporte Línea 61

Proyecto linea57_control · Aplicación web Laravel

Campo	Detalle
Sistema o módulo	Asignación de turno (app/Http/Controllers/AsignacionTurnoController.php)
Función evaluada	AsignacionTurnoController::store (Store)
Tipo de operación	Transaccional: creación de una asignación diaria de conductor, micro, ruta y turno
Tecnología	Laravel 13 · PHP 8.3 · MySQL 8 (producción) · SQLite en memoria (pruebas)
Métodos aplicados	Caja blanca: instrucciones, decisiones, condiciones y caminos. Caja negra: equivalencia, valores límite, tabla de decisión y transición de estados.
Casos diseñados y ejecutados	49 casos automatizados · 149 aserciones
Resultado global	49 aprobados · 0 fallidos
Fecha de ejecución	5 de noviembre de 2026

Documento elaborado a partir de la ejecución real de la suite de pruebas sobre la ruta HTTP del sistema. Los resultados obtenidos que se registran en las tablas provienen de esa ejecución.

Contenido

1. Resumen ejecutivo
2. Alcance, operación seleccionada y justificación
3. Código de la función seleccionada: store
4. Parte I. Pruebas de caja blanca
4.1 Método 1. Cobertura de instrucciones
4.2 Método 2. Cobertura de decisiones o ramas
4.3 Método 3. Cobertura de condiciones
4.4 Método 4. Cobertura de caminos
5. Parte II. Pruebas de caja negra
5.1 Método 1. Partición de equivalencia
5.2 Método 2. Análisis de valores límite
5.3 Método 3. Tabla de decisión
5.4 Método 4. Transición de estados
6. Parte III. Registro general de casos y evidencia de ejecución
7. Conclusiones, observaciones y recomendaciones
A. Anexo A. Casos automatizados y entorno de ejecución
B. Anexo B. Diferencia entre caja blanca y caja negra

Las tablas conservan la estructura de las plantillas del documento de referencia. Cuando un método requiere más de dos casos, la plantilla se amplía con filas adicionales para registrar todos los caminos y resultados de la ejecución.

1. Resumen ejecutivo

El presente informe aplica los métodos de prueba de caja blanca y caja negra descritos en el documento «Pruebas de caja blanca y caja negra» al sistema linea57_control, módulo de asignación de turnos. La operación transaccional seleccionada es la asignación de turno y la función elegida es store, implementada en AsignacionTurnoController::store. El informe analiza esa función como unidad de prueba; el servicio que invoca se considera una dependencia y no se transcribe ni se analiza por separado.

Se eligió store porque, aun siendo una función breve, concentra dos puntos de decisión claros: la validación del request y la invocación del servicio que materializa la regla de negocio. La validación del request contiene reglas obligatorias, de tipo, de existencia y de longitud que generan ramas verdaderas y falsas; la llamada al servicio puede completarse o propagar un error de negocio. Esa combinación produce distintos caminos lógicos y condiciones que los métodos de prueba permiten analizar, sin necesidad de evaluar otras funciones del módulo.

Se diseñaron y ejecutaron 49 casos automatizados sobre la ruta HTTP real del sistema, con una base de datos SQLite en memoria y el trait RefreshDatabase de Laravel. Todos los casos finalizaron correctamente: 49 aprobados, 0 fallidos y 149 aserciones verificadas. Las tablas de cada método registran el resultado esperado, el resultado obtenido y el estado de la prueba.

Indicadores de la ejecución

Indicador	Valor
Casos de caja blanca	32 (instrucciones: 7 · decisiones: 11 · condiciones: 6 · caminos: 8)
Casos de caja negra	17 (equivalencia: 3 · valores límite: 6 · tabla de decisión: 4 · estados: 4)
Total de casos	49
Casos aprobados	49
Casos fallidos	0
Aserciones ejecutadas	149
Tiempo aproximado de la suite	60,17 s
Cobertura de instrucciones del camino Store	100 % de las instrucciones ejecutables del flujo de creación
Cobertura de decisiones	100 % de las ramas identificadas
Cobertura de condiciones simples	100 % (cada condición tomó verdadero y falso al menos una vez)
Cobertura de caminos	9 caminos factibles identificados y ejecutados

2. Alcance, operación seleccionada y justificación

El alcance de este informe es la operación transaccional de asignación de turno, específicamente la creación de una asignación desde el panel web. No se evalúan en este documento la edición, el inicio, la finalización ni la cancelación de asignaciones, salvo cuando se usan como precondición para probar el estado inicial de Store. La recepción de puntos GPS y el control de recorrido pertenecen a otras operaciones transaccionales del sistema y pueden abordarse en un ciclo de pruebas complementario.

2.1. Datos de la operación

Aspecto	Detalle
Transacción	Asignación de turno
Función	store (crear una asignación)
Controlador	app/Http/Controllers/AsignacionTurnoController.php, método store (líneas 106-127)
Dependencia invocada	AsignacionTurnoService::crear($data); no se analiza como función independiente

Aspecto	Detalle
Modelo	app/Models/AsignacionTurno.php, tabla asignacion_turnos
Ruta	POST /asignacion-turno (Route::resource('asignacion-turno', ...))
Middleware	auth · verified · ensure.role.assigned · role:admin|fiscalizador|propietario
Entradas	fecha, turno_id, ruta_id, micro_id, conductor_id, observaciones
Salida exitosa	Registro en asignacion_turnos con estado «pendiente» y redirección a asignacion-turno.index
Salida fallida	Redirección con errores de validación; no se crea ningún registro

2.2. Reglas de negocio verificadas

- El turno seleccionado debe existir y estar en estado «activo».
- El micro seleccionado debe existir y estar en estado «activo».
- El conductor seleccionado debe existir y estar en estado «activo».
- La ruta seleccionada debe existir y estar en estado «activo».
- No debe existir otra asignación no cancelada del mismo conductor para la misma fecha y turno.
- No debe existir otra asignación no cancelada del mismo micro para la misma fecha y turno.
- Una asignación cancelada no bloquea la creación de una nueva asignación para el mismo conductor, micro, fecha y turno.
- El campo observaciones es opcional y admite como máximo 1000 caracteres.
- Al crear, el estado siempre es «pendiente»; hora_salida y hora_llegada no se toman del formulario.

2.3. Justificación de la selección

Store es una función adecuada para la prueba de caja blanca porque su lógica no es lineal: contiene una validación con varias reglas encadenadas, una invocación a una dependencia que puede fallar y una respuesta final. También es adecuada para la caja negra porque cada regla se puede expresar como clase de equivalencia, valor límite o combinación de decisión, y el resultado se observa en la respuesta HTTP y en la base de datos. Una función con un solo camino lineal no permitiría demostrar la cobertura de ramas con la misma claridad.

2.4. Flujo de ejecución

Paso	Acción ejecutada	Resultado
1	El usuario autenticado envía el formulario a POST /asignacion-turno.	La petición pasa por los middleware de sesión, correo verificado y rol.
2	$request->validate([...]) valida fecha, turno_id, ruta_id, micro_id, conductor_id y observaciones.	Si una regla falla, Laravel redirige con errores y no se ejecuta la línea siguiente.
3	La validación es superada y store invoca a $this->asignacionTurnoService->crear($data).	El servicio aplica las reglas de negocio; store no replica esa lógica.
4	El servicio completa la creación del registro en estado «pendiente».	Se crea una fila en asignacion_turnos.
5	Si el servicio detecta una regla de negocio incumplida, lanza una ValidationException.	La excepción se propaga y el usuario recibe el error; no se crea el registro.
6	Store retorna redirect()->route('asignacion-turno.index')->with('success', ...).	El usuario ve la confirmación y la nueva asignación en el listado.

3. Código de la función seleccionada: store

A continuación se transcribe únicamente la función store del controlador, que es la unidad de análisis de este informe. La función recibe la petición, valida los campos con las reglas de Laravel, delega la creación en el servicio y redirige al listado. Los números de línea corresponden al archivo original del proyecto.

El servicio AsignacionTurnoService es una dependencia de store. Se menciona porque store lo invoca, pero no se transcribe ni se analiza como función independiente: las reglas de negocio que aplica se observan a través del resultado de store, es decir, de la respuesta HTTP y del estado de la base de datos.

Código 1. AsignacionTurnoController.php, líneas 106-127. Función store: validación del request, llamada al servicio y redirección.

```
106      /**
107       * Almacena una nueva asignación de turno.
108       * SDD Sección 13 y 40: estado siempre 'pendiente' al crear;
109       * hora_salida y hora_llegada son determinados por el flujo de ejecución.
110       */
111      public function store(Request $request): RedirectResponse
112      {
113          $data = $request->validate([
114              'fecha' => ['required', 'date'],
115              'turno_id' => ['required', 'integer', 'exists:turno,id'],
116              'ruta_id' => ['required', 'integer', 'exists:ruta,id'],
117              'micro_id' => ['required', 'integer', 'exists:micro,id'],
118              'conductor_id' => ['required', 'integer', 'exists:conductor,id'],
119              'observaciones' => ['nullable', 'string', 'max:1000'],
120          ]);
121
122          $this->asignacionTurnoService->crear($data);
123
124          return redirect()
125              ->route('asignacion-turno.index')
126              ->with('success', 'Asignación de turno creada correctamente.');
127      }
```

Lectura del código: store tiene tres instrucciones ejecutables principales. La primera, $request->validate([...]), concentra las reglas de entrada y puede generar una redirección con errores. La segunda, $this->asignacionTurnoService->crear($data), delega la creación y puede propagar una ValidationException de negocio. La tercera, el return redirect()->route(...)->with('success', ...), solo se ejecuta cuando las dos anteriores terminan sin excepción.

4. Parte I. Pruebas de caja blanca

La caja blanca examina la estructura interna de la función. Para este informe se analizaron las instrucciones de store, las decisiones que introduce la validación del request y la invocación del servicio, las condiciones simples que componen las reglas de validación y los caminos que puede recorrer la ejecución desde la recepción de la petición hasta la respuesta final. Los casos se ejecutaron contra la ruta HTTP real, de modo que la cobertura incluye la validación del controlador, la delegación en el servicio y la respuesta al usuario.

4.1. Método 1. Cobertura de instrucciones

Objetivo: ejecutar al menos una vez cada instrucción ejecutable de store. La plantilla se amplió a siete casos porque la función combina la validación del request, la llamada al servicio y la redirección, y cada una de esas instrucciones puede observarse en un resultado distinto.

Tabla 4.1. Plantilla llenada: cobertura de instrucciones.

Campo	Respuesta del estudiante
ID de prueba	CB-I01
Sistema o módulo	Asignación de turno · función store
Objetivo	Comprobar que se ejecute al menos una vez cada instrucción del camino de creación y de cada salida de error.
Instrucciones que se probarán	$request->validate([...]) con las reglas de todos los campos; $this->asignacionTurnoService->crear($data); return redirect()->route('asignacion-turno.index')->with('success', ...). Las reglas de negocio que aplica el servicio se observan como resultado de la llamada.
Caso de prueba 1	Datos válidos: fecha actual, turno activo, ruta activa, micro activo y conductor activo, sin asignaciones previas.
Resultado esperado	Crear la asignación con estado «pendiente» y redirigir a asignacion-turno.index.
Caso de prueba 2	Turno existente con estado «inactivo».
Resultado esperado	Rechazar la creación con error en turno_id.
Caso de prueba 3	Micro existente con estado «inactivo».
Resultado esperado	Rechazar la creación con error en micro_id.
Caso de prueba 4	Conductor existente con estado «inactivo».
Resultado esperado	Rechazar la creación con error en conductor_id.
Caso de prueba 5	Ruta existente con estado «inactivo».
Resultado esperado	Rechazar la creación con error en ruta_id.
Caso de prueba 6	Todos los recursos activos, pero ya existe una asignación no cancelada del mismo conductor para la misma fecha y turno.
Resultado esperado	Rechazar la creación con error en conductor_id.
Caso de prueba 7	Todos los recursos activos, conductor libre, pero ya existe una asignación no cancelada del mismo micro para la misma fecha y turno.
Resultado esperado	Rechazar la creación con error en micro_id.
Resultado obtenido	Los siete casos se ejecutaron. El caso 1 recorrió las tres instrucciones de store y creó una asignación «pendiente»; los casos 2 a 7 se detuvieron en la validación o en la llamada al servicio, no crearon registros y devolvieron el error del campo correspondiente. Todas las instrucciones de store fueron ejecutadas al menos una vez.
Estado	Aprobada

Tabla 4.2. Matriz de correspondencia instrucción → caso de store.

Instrucción del código	Caso que la ejecuta	¿Cubierta?
$request->validate([...])	CB-I01 a CB-I07	Sí
Regla 'fecha' => required|date	CB-I01 (válida) y CB-P08 (ausente)	Sí
Regla 'turno_id' => required|integer|exists	CB-I01 (válida) y CB-I02 (turno inactivo)	Sí
Regla 'ruta_id' => required|integer|exists	CB-I01 (válida) y CB-I05 (ruta inactiva)	Sí
Regla 'micro_id' => required|integer|exists	CB-I01 (válida) y CB-I03 (micro inactivo)	Sí
Regla 'conductor_id' => required|integer|exists	CB-I01 (válida) y CB-I04 (conductor inactivo)	Sí
Regla 'observaciones' => nullable|string|max:1000	CN-VL01 a CN-VL06	Sí
$this->asignacionTurnoService->crear($data)	CB-I01 a CB-I07	Sí
return redirect()->route('asignacion-turno.index')->with('success', ...)	CB-I01, CN-TE01, CN-TE04	Sí

Conclusión del método: la cobertura de instrucciones de store es del 100 %. Las tres instrucciones principales y todas las reglas declaradas en la validación del request fueron ejecutadas al menos una vez. El servicio se considera una dependencia: sus decisiones internas no se contabilizan aquí como instrucciones de store, pero su resultado se verifica en la respuesta de la función.

4.2. Método 2. Cobertura de decisiones o ramas

Objetivo: comprobar cada resultado posible de las decisiones que intervienen en store. Se distinguen dos niveles: las reglas declaradas en la validación del request, cada una de las cuales puede aceptar o rechazar el dato, y la invocación del servicio, que puede completar la creación o propagar un error de negocio. La plantilla se presenta primero para la decisión principal y luego se consolida el resto en una tabla de ramas.

Tabla 4.3. Plantilla llenada: decisión «¿la validación del request es superada?».

Campo	Respuesta del estudiante
ID de prueba	CB-D01
Sistema o módulo	Asignación de turno · función store
Objetivo	Verificar las dos ramas de la decisión que controla si store invoca o no al servicio.
Condición que se evaluará	¿Todos los campos del request cumplen las reglas de validación?
Caso de prueba 1	Request válido: fecha, turno_id, ruta_id, micro_id y conductor_id presentes y con formato correcto; observaciones dentro del límite.
Rama recorrida	Verdadera (continúa hacia la llamada al servicio).
Resultado esperado	Invocar al servicio y crear la asignación en estado «pendiente».
Caso de prueba 2	Request inválido: falta fecha o algún identificador, o un identificador no existe en el catálogo.
Rama recorrida	Falsa (redirección con errores de validación).
Resultado esperado	Rechazar la petición sin invocar al servicio y sin crear registros.
Resultado obtenido	El caso 1 creó una asignación «pendiente» y redirigió al listado; el caso 2 no invocó al servicio y devolvió los errores de los campos afectados.
Estado	Aprobada

Tabla 4.4. Consolidado de decisiones y ramas ejecutadas.

Decisión	Caso rama verdadera	Resultado	Caso rama falsa	Resultado	Estado
D1. ¿La validación del request es superada?	Datos válidos · CB-D01-V	Invoca al servicio	Campo inválido · CB-D01-F	Errores de validación	Aprobada
D2. ¿turno_id es requerido, entero y existente?	turno_id válido · CB-D02-V	Continúa	turno_id = 999999 · CB-D02-F	Error turno_id	Aprobada
D3. ¿ruta_id es requerido, entero y existente?	ruta_id válido · CB-D03-V	Continúa	ruta_id = 999999 · CB-D03-F	Error ruta_id	Aprobada
D4. ¿micro_id es requerido, entero y existente?	micro_id válido · CB-D04-V	Continúa	micro_id = 999999 · CB-D04-F	Error micro_id	Aprobada
D5. ¿conductor_id es requerido, entero y existente?	conductor_id válido · CB-D05-V	Continúa	conductor_id = 999999 · CB-D05-F	Error conductor_id	Aprobada
D6. ¿observaciones respeta max:1000?	Observaciones de 1000 caracteres · CB-D06-V	Continúa	Observaciones de 1001 caracteres · CB-D06-F	Error observaciones	Aprobada
D7. ¿El servicio completa la creación?	Todos los recursos activos · CB-D07-V	Crea la asignación	Recurso inactivo o inexistente · CB-D07-F	Error del recurso	Aprobada
D8. ¿El conductor ya está ocupado?	Conductor ocupado · CB-D08-V	Error conductor_id	Conductor libre · CB-D08-F	Crea la asignación	Aprobada
D9. ¿El micro ya está ocupado?	Micro ocupado · CB-D09-V	Error micro_id	Micro libre · CB-D09-F	Crea la asignación	Aprobada

Resultado obtenido: las nueve decisiones recorrieron sus dos ramas. D1 a D6 corresponden a la validación del request y se resuelven antes de la llamada al servicio. D8 y D9 corresponden a reglas de negocio que store observa a través del servicio; en ambas la rama verdadera produce el rechazo y la falsa permite la creación. No quedaron ramas sin cubrir.

4.3. Método 3. Cobertura de condiciones

Store declara condiciones compuestas en las reglas de validación de los identificadores: «required|integer|exists:tabla,id» para turno_id, ruta_id, micro_id y conductor_id. Cada regla se descompone en dos condiciones simples: A = el campo está presente; B = el valor es un entero que existe en el catálogo. La tabla registra casos en los que A y B toman verdadero y falso al menos una vez para micro, conductor y ruta.

Tabla 4.5. Plantilla llenada: cobertura de condiciones de las reglas de validación.

ID	Validación	Dato de entrada	Condición A (presente)	Condición B (válido)	Resultado esperado	Resultado obtenido	Estado
CB-C01	Micro	micro_id válido	Verdadero	Verdadero	Crear asignación	Creada «pendiente»	Aprobada
CB-C02	Micro	micro_id ausente	Falso	No evaluada	Error micro_id	Error micro_id	Aprobada
CB-C03	Micro	micro_id = 999999	Verdadero	Falso	Error micro_id	Error micro_id	Aprobada
CB-C04	Conductor	conductor_id válido	Verdadero	Verdadero	Crear asignación	Creada «pendiente»	Aprobada
CB-C05	Conductor	conductor_id ausente	Falso	No evaluada	Error conductor_id	Error conductor_id	Aprobada
CB-C06	Conductor	conductor_id = 999999	Verdadero	Falso	Error conductor_id	Error conductor_id	Aprobada
CB-C07	Ruta	ruta_id válido	Verdadero	Verdadero	Crear asignación	Creada «pendiente»	Aprobada
CB-C08	Ruta	ruta_id ausente	Falso	No evaluada	Error ruta_id	Error ruta_id	Aprobada
CB-C09	Ruta	ruta_id = 999999	Verdadero	Falso	Error ruta_id	Error ruta_id	Aprobada

Análisis: la condición A (campo presente) tomó verdadero en CB-C01, CB-C03, CB-C04, CB-C06, CB-C07 y CB-C09, y falso en CB-C02, CB-C05 y CB-C08. La condición B (identificador válido y existente) tomó verdadero en CB-C01, CB-C04 y CB-C07, y falso en CB-C03, CB-C06 y CB-C09. Por lo tanto, cada condición simple fue verdadera y falsa al menos una vez. Cuando A es falsa, B no se evalúa por el cortocircuito de la validación, lo cual es el comportamiento esperado; la cobertura de B se logra con los casos de identificador inexistente. La cobertura de condiciones para las tres reglas de validación queda satisfecha.

> Observación técnica: la regla exists evita que store invoque al servicio con un identificador que no pertenece a ningún catálogo. La prueba CB-C02, CB-C05 y CB-C08 confirma que el sistema responde con un error controlado antes de la llamada al servicio.

4.4. Método 4. Cobertura de caminos

Los caminos se identificaron a partir de la secuencia que sigue store: validación del request, llamada al servicio y redirección. Como la validación y el servicio detienen el flujo en el primer fallo, se obtiene un camino exitoso y varias salidas tempranas. También se incluyó el camino de error de la validación del controlador y el camino en el que existe una asignación cancelada, porque cambia el resultado de la llamada.

Diagrama de caminos de la función store.

```
P1  Request válido → turno activo → micro válido → conductor válido → ruta válida
    → conductor libre → micro libre → crear «pendiente» → redirección con éxito
P2  Request válido → turno inactivo → error turno_id
P3  Request válido → turno activo → micro inexistente o inactivo → error micro_id
P4  Request válido → turno y micro válidos → conductor inexistente o inactivo → error conductor_id
P5  Request válido → turno, micro y conductor válidos → ruta inexistente o inactiva → error ruta_id
P6  Recursos válidos → conductor ocupado en la misma fecha y turno → error conductor_id
P7  Recursos válidos y conductor libre → micro ocupado en la misma fecha y turno → error micro_id
P8  Request incompleto → errores de validación de campos obligatorios
P9  Recursos válidos → existe una asignación cancelada del mismo conductor y turno
    → la cancelada no bloquea → crear una nueva asignación «pendiente»
```

Tabla 4.6. Plantilla llenada: cobertura de caminos.

Camino	Descripción del recorrido	Datos de prueba	Resultado esperado	Resultado obtenido	Estado
P1	Request válido → turno activo → micro válido → conductor válido → ruta válida → conductor libre → micro libre → crear «pendiente» → redirección.	Todos los recursos activos, sin asignaciones previas.	Asignación creada en estado «pendiente».	1 registro creado; redirección a asignacion-turno.index.	Aprobada
P2	Request válido → turno inactivo → error.	turno.estado = inactivo.	Error en turno_id; no se crea registro.	Error en turno_id; 0 registros.	Aprobada
P3	Request válido → turno activo → micro inexistente o inactivo → error.	micro.estado = inactivo / micro_id = 999999.	Error en micro_id; no se crea registro.	Error en micro_id; 0 registros.	Aprobada
P4	Request válido → turno y micro válidos → conductor inexistente o inactivo → error.	conductor.estado = inactivo / conductor_id = 999999.	Error en conductor_id; no se crea registro.	Error en conductor_id; 0 registros.	Aprobada
P5	Request válido → turno, micro y conductor válidos → ruta inexistente o inactiva → error.	ruta.estado = inactivo / ruta_id = 999999.	Error en ruta_id; no se crea registro.	Error en ruta_id; 0 registros.	Aprobada
P6	Todos los recursos válidos → conductor ya ocupado en la misma fecha y turno → error.	Asignación previa no cancelada del mismo conductor.	Error en conductor_id; no se crea un segundo registro.	Error en conductor_id; 1 registro previo.	Aprobada
P7	Todos los recursos válidos y conductor libre → micro ya ocupado en la misma fecha y turno → error.	Asignación previa no cancelada del mismo micro.	Error en micro_id; no se crea un segundo registro.	Error en micro_id; 1 registro previo.	Aprobada
P8	Request incompleto → validación del controlador → errores de campos obligatorios.	Petición sin fecha, turno_id, ruta_id, micro_id ni conductor_id.	Errores en los cinco campos; no se llama al servicio.	Errores en los cinco campos; 0 registros.	Aprobada
P9	Todos los recursos válidos → existe una asignación cancelada del mismo conductor y turno → la cancelada no bloquea → crear una nueva «pendiente».	Asignación previa en estado «cancelado» y mismo conductor, fecha y turno.	Se crea una nueva asignación «pendiente»; la cancelada se conserva.	2 registros: 1 cancelada y 1 pendiente.	Aprobada

Resultado obtenido: se ejecutaron los nueve caminos factibles. El camino P1 y el camino adicional P9 terminan en una creación; los caminos P2 a P8 terminan en un rechazo controlado sin dejar registros parciales. La cobertura de caminos de la función store queda satisfecha para los caminos identificados.

5. Parte II. Pruebas de caja negra

La caja negra verifica el comportamiento externo del sistema a partir de las reglas de negocio y de los datos de entrada, sin depender de la estructura interna. Los casos se ejecutaron sobre la misma ruta HTTP y se observaron la respuesta, los mensajes de validación y el estado de la base de datos. Se aplicaron los cuatro métodos de la guía: partición de equivalencia, análisis de valores límite, tabla de decisión y transición de estados.

5.1. Método 1. Partición de equivalencia

Se tomó como dato evaluado el estado del micro, porque la regla es representativa de los tres recursos que deben existir y estar activos. Las clases identificadas son: micro activo (válida), micro inactivo (inválida) y micro inexistente (inválida).

Tabla 5.1. Plantilla llenada: partición de equivalencia.

Campo	Respuesta del estudiante
ID de prueba	CN-PE01
Sistema o módulo	Asignación de turno · función store
Dato evaluado	Estado y existencia del micro
Regla del sistema	El micro debe existir y estar en estado «activo» para poder asignarlo.
Clase válida	Micro existente con estado «activo».
Caso de prueba válido	micro_id = ID de un micro activo; turno, conductor y ruta activos.
Resultado esperado	Aceptar y crear la asignación en estado «pendiente».
Clase inválida 1	Micro existente con estado «inactivo».
Caso de prueba	micro_id = ID de un micro inactivo.
Resultado esperado	Rechazar con error en micro_id.
Clase inválida 2	Micro inexistente.
Caso de prueba	micro_id = 999999.
Resultado esperado	Rechazar con error en micro_id.
Resultado obtenido	La clase válida creó una asignación «pendiente». Las dos clases inválidas fueron rechazadas con el mensaje «El micro seleccionado no está activo o no existe» o con el error de existencia, según el caso. No se crearon registros en las clases inválidas.
Estado	Aprobada

La misma partición se aplicó a conductor, ruta y turno. Los casos CB-C04 a CB-C09 y CB-D04 a CB-D07 verifican las clases equivalentes de esos recursos; el resultado fue el mismo comportamiento controlado.

5.2. Método 2. Análisis de valores límite

El campo numérico con un límite explícito en Store es observaciones: es opcional y admite un máximo de 1000 caracteres. Se probaron el límite inferior (0 caracteres), valores intermedios y el límite superior (1000), además del valor inmediatamente superior (1001).

Tabla 5.2. Plantilla llenada: análisis de valores límite de observaciones.

ID	Valor ingresado	Tipo de valor	Resultado esperado	Resultado obtenido	Estado
CN-VL01	Sin observaciones (null)	Límite inferior: 0 caracteres	Aceptar	Creada con observaciones = null	Aprobada
CN-VL02	Cadena vacía «»	Límite inferior: 0 caracteres	Aceptar	Creada con observaciones = «»	Aprobada
CN-VL03	«A»	Dentro del rango: 1 carácter	Aceptar	Creada con observaciones = «A»	Aprobada
CN-VL04	999 caracteres	Cerca del límite superior, dentro del rango	Aceptar	Creada con 999 caracteres	Aprobada

ID	Valor ingresado	Tipo de valor	Resultado esperado	Resultado obtenido	Estado
CN-VL05	1000 caracteres	Límite superior	Aceptar	Creada con 1000 caracteres	Aprobada
CN-VL06	1001 caracteres	Encima del límite superior	Rechazar	Error de validación en observaciones; 0 registros	Aprobada

Resultado obtenido: el sistema aceptó el campo ausente, la cadena vacía, un carácter, 999 y 1000 caracteres. Rechazó 1001 caracteres con un error de validación y no creó el registro. El límite superior está correctamente implementado y no se observó un desplazamiento de uno (off-by-one).

5.3. Método 3. Tabla de decisión

Para probar la combinación de reglas se agruparon las condiciones en dos: C1 = todos los recursos están activos; C2 = no existe duplicidad de conductor ni de micro en la misma fecha y turno. La tabla contiene las cuatro combinaciones posibles y la acción esperada para cada una.

Tabla 5.3. Tabla de decisión: recursos activos y disponibilidad.

Regla	C1. Recursos activos	C2. Sin duplicidad	Acción esperada
1	Sí	Sí	Crear la asignación en estado «pendiente».
2	Sí	No	Mostrar error de duplicidad: conductor_id o micro_id, según corresponda.
3	No	Sí	Mostrar error del recurso inactivo o inexistente.
4	No	No	Mostrar error del recurso; la validación del recurso se ejecuta antes de la duplicidad.

Tabla 5.4. Casos concretos de la tabla de decisión.

ID	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CN-TD01	Todos los recursos activos; conductor y micro libres en la fecha y turno.	Crear asignación «pendiente».	1 registro creado en estado «pendiente».	Aprobada
CN-TD02	Todos los recursos activos; ya existe una asignación no cancelada del mismo conductor en la fecha y turno.	Error en conductor_id por duplicidad.	Error en conductor_id; 1 registro previo.	Aprobada
CN-TD03	Micro inactivo; conductor libre.	Error en micro_id por recurso inactivo.	Error en micro_id; 0 registros.	Aprobada
CN-TD04	Micro inactivo y conductor duplicado al mismo tiempo.	Error en micro_id; el recurso se valida antes de la duplicidad.	Error en micro_id; no se informa conductor_id; 1 registro previo.	Aprobada
CN-TD05	Todos los recursos activos; conductor libre; micro duplicado en la fecha y turno.	Error en micro_id por duplicidad.	Error en micro_id; 1 registro previo.	Aprobada

Resultado obtenido: las cuatro reglas de la tabla de decisión fueron cubiertas. La regla 4 confirmó el orden de las validaciones: cuando hay un recurso inactivo y una duplicidad simultáneos, el sistema informa primero el recurso inactivo. Este comportamiento es coherente con el código y con la expectativa de negocio de no evaluar la disponibilidad de un recurso que no es utilizable.

5.4. Método 4. Transición de estados

Para la función store, los estados observables son: «sin asignación», «pendiente» y «cancelada». La acción Store puede crear una asignación pendiente, ser rechazada sin cambiar el estado, o crear una nueva asignación cuando la existente está cancelada. La tabla registra las transiciones válidas y las acciones rechazadas.

Tabla 5.5. Plantilla llenada: transición de estados de la operación Store.

ID	Estado inicial	Acción realizada	Resultado esperado	Estado final esperado	Resultado obtenido	Estado de la prueba
CN-TE01	Sin asignación	Store con datos válidos: turno, ruta, micro y conductor activos, sin duplicidad.	Confirmar la creación.	Asignación en estado «pendiente».	1 registro en estado «pendiente»; redirección con éxito.	Aprobada
CN-TE02	Sin asignación	Store con turno inactivo.	Mostrar error y no crear.	Sin asignación.	0 registros; error en turno_id.	Aprobada
CN-TE03	Asignación pendiente existente	Store con el mismo conductor, fecha y turno; micro diferente.	Rechazar por duplicidad.	La asignación existente permanece «pendiente»; no se crea otra.	1 registro; estado «pendiente» sin cambios; error en conductor_id.	Aprobada
CN-TE04	Asignación cancelada existente	Store con el mismo conductor, fecha y turno; micro diferente.	Crear una nueva asignación.	Una asignación «cancelada» y una nueva «pendiente».	2 registros: la cancelada se conserva y la nueva queda «pendiente».	Aprobada

Diagrama de transición de estados observado.

```
[Sin asignación] ──Store válido──► [Pendiente]
[Sin asignación] ──Store inválido──► [Sin asignación]   (rechazo, no hay registro)
[Pendiente]      ──Store duplicado──► [Pendiente]       (rechazo, se conserva el registro)
[Cancelada]      ──Store válido──► [Pendiente]          (nueva asignación; la cancelada se conserva)
```

Resultado obtenido: las transiciones válidas produjeron el estado final esperado y las acciones rechazadas no alteraron el estado existente. La asignación cancelada no bloquea la creación de una nueva, lo que confirma que store obtiene del servicio el resultado correcto también cuando existe historial cancelado.

6. Parte III. Registro general de casos y evidencia de ejecución

El registro general consolida los 49 casos ejecutados. Los casos se identifican con el prefijo CB para caja blanca y CN para caja negra. Todos fueron ejecutados mediante la suite automatizada y todos finalizaron en estado «Aprobada».

Tipo de prueba	Método	Casos	Aprobados	Fallidos
Caja blanca	Cobertura de instrucciones	7	7	0
Caja blanca	Cobertura de decisiones o ramas	11	11	0
Caja blanca	Cobertura de condiciones	6	6	0
Caja blanca	Cobertura de caminos	8	8	0
Caja negra	Partición de equivalencia	3	3	0
Caja negra	Análisis de valores límite	6	6	0
Caja negra	Tabla de decisión	4	4	0
Caja negra	Transición de estados	4	4	0
Total		49	49	0

Tabla 6.1. Registro general de casos ejecutados.

Código	Tipo	Método	Objetivo	Resultado
CB-I01	Caja blanca	Instrucciones	Camino exitoso: todas las instrucciones del flujo.	Conforme
CB-I02	Caja blanca	Instrucciones	Turno inactivo: instrucción de rechazo.	Conforme
CB-I03	Caja blanca	Instrucciones	Micro inactivo: instrucción de rechazo.	Conforme
CB-I04	Caja blanca	Instrucciones	Conductor inactivo: instrucción de rechazo.	Conforme
CB-I05	Caja blanca	Instrucciones	Ruta inactiva: instrucción de rechazo.	Conforme
CB-I06	Caja blanca	Instrucciones	Conductor duplicado: instrucción de rechazo.	Conforme
CB-I07	Caja blanca	Instrucciones	Micro duplicado: instrucción de rechazo.	Conforme
CB-D01-V	Caja blanca	Decisiones	Decisión turno activo, rama verdadera.	Conforme
CB-D01-F	Caja blanca	Decisiones	Decisión turno activo, rama falsa.	Conforme
CB-D02-F	Caja blanca	Decisiones	Decisión micro existe, rama falsa.	Conforme
CB-D03-F	Caja blanca	Decisiones	Decisión micro activo, rama falsa.	Conforme
CB-D04-F	Caja blanca	Decisiones	Decisión conductor existe, rama falsa.	Conforme
CB-D05-F	Caja blanca	Decisiones	Decisión conductor activo, rama falsa.	Conforme
CB-D06-F	Caja blanca	Decisiones	Decisión ruta existe, rama falsa.	Conforme
CB-D07-F	Caja blanca	Decisiones	Decisión ruta activa, rama falsa.	Conforme
CB-D08-V	Caja blanca	Decisiones	Conductor ocupado, rama verdadera.	Conforme
CB-D08-F	Caja blanca	Decisiones	Conductor libre, rama falsa.	Conforme
CB-D09-V	Caja blanca	Decisiones	Micro ocupado, rama verdadera.	Conforme
CB-C01	Caja blanca	Condiciones	Micro existente y activo: A=V, B=V.	Conforme
CB-C02	Caja blanca	Condiciones	Micro inexistente: A=F, B no evaluada.	Conforme
CB-C03	Caja blanca	Condiciones	Micro inactivo: A=V, B=F.	Conforme
CB-C04	Caja blanca	Condiciones	Conductor existente y activo: A=V, B=V.	Conforme
CB-C05	Caja blanca	Condiciones	Conductor inexistente: A=F, B no evaluada.	Conforme
CB-C06	Caja blanca	Condiciones	Conductor inactivo: A=V, B=F.	Conforme
CB-P01	Caja blanca	Caminos	Camino exitoso sin duplicidad.	Conforme
CB-P02	Caja blanca	Caminos	Camino turno inactivo.	Conforme
CB-P03	Caja blanca	Caminos	Camino micro inválido.	Conforme
CB-P04	Caja blanca	Caminos	Camino conductor inválido.	Conforme
CB-P05	Caja blanca	Caminos	Camino ruta inválida.	Conforme
CB-P06	Caja blanca	Caminos	Camino conductor duplicado.	Conforme
CB-P07	Caja blanca	Caminos	Camino micro duplicado.	Conforme
CB-P08	Caja blanca	Caminos	Camino de error de campos obligatorios.	Conforme

Código	Tipo	Método	Objetivo	Resultado
CN-PE01	Caja negra	Equivalencia	Clase válida: micro activo.	Conforme
CN-PE02	Caja negra	Equivalencia	Clase inválida: micro inactivo.	Conforme
CN-PE03	Caja negra	Equivalencia	Clase inválida: micro inexistente.	Conforme
CN-VL01	Caja negra	Valores límite	Observaciones ausente: límite inferior 0.	Conforme
CN-VL02	Caja negra	Valores límite	Observaciones vacía: límite inferior 0.	Conforme
CN-VL03	Caja negra	Valores límite	Observaciones de 1 carácter: dentro del rango.	Conforme
CN-VL04	Caja negra	Valores límite	Observaciones de 999 caracteres: cerca del límite superior.	Conforme
CN-VL05	Caja negra	Valores límite	Observaciones de 1000 caracteres: límite superior.	Conforme
CN-VL06	Caja negra	Valores límite	Observaciones de 1001 caracteres: encima del límite.	Conforme
CN-TD01	Caja negra	Tabla de decisión	Recursos activos y sin duplicidad: crea.	Conforme
CN-TD02	Caja negra	Tabla de decisión	Recursos activos con duplicidad: rechaza.	Conforme
CN-TD03	Caja negra	Tabla de decisión	Recurso inactivo sin duplicidad: rechaza.	Conforme
CN-TD04	Caja negra	Tabla de decisión	Recurso inactivo con duplicidad: prevalece el recurso.	Conforme
CN-TE01	Caja negra	Transición de estados	Sin asignación + Store válido: pasa a pendiente.	Conforme
CN-TE02	Caja negra	Transición de estados	Sin asignación + Store inválido: permanece sin asignación.	Conforme
CN-TE03	Caja negra	Transición de estados	Pendiente + Store duplicado: permanece pendiente.	Conforme
CN-TE04	Caja negra	Transición de estados	Cancelada + Store válido: crea nueva pendiente.	Conforme

6.1. Evidencia de ejecución

La suite se ejecutó con el comando de pruebas de Laravel. El archivo de pruebas se encuentra en tests/Feature/PruebasCajaBlancaNegraAsignacionTurnoTest.php y contiene un método por caso, con aserciones sobre la redirección, los errores de validación y el estado de la base de datos.

Evidencia de la ejecución automatizada.

```
php artisan test tests/Feature/PruebasCajaBlancaNegraAsignacionTurnoTest.php

PASS  Tests\Feature\PruebasCajaBlancaNegraAsignacionTurnoTest
  Tests:    49 passed (149 assertions)
  Duration: 60.17s
```

Elemento	Detalle
Entorno de ejecución	Windows · PHP 8.3 · Laravel 13 · PHPUnit 12
Base de datos de prueba	SQLite en memoria (DB_CONNECTION=sqlite, DB_DATABASE=:memory:)
Aislamiento	RefreshDatabase: cada caso parte de una base limpia
Nivel de prueba	Prueba de integración HTTP sobre la ruta POST /asignacion-turno
Archivo de pruebas	tests/Feature/PruebasCajaBlancaNegraAsignacionTurnoTest.php
Resultado	49 casos aprobados · 0 fallidos · 149 aserciones

7. Conclusiones, observaciones y recomendaciones

7.1. Conclusiones

- La función store cumple las reglas de negocio verificadas: solo crea asignaciones cuando el turno, el micro, el conductor y la ruta existen y están activos, y cuando no hay duplicidad de conductor o micro en la misma fecha y turno.
- La cobertura de instrucciones de store es del 100 %: la validación del request, la llamada al servicio y la redirección se ejecutaron al menos una vez.
- Las nueve decisiones identificadas recorrieron sus ramas verdadera y falsa. No quedaron ramas sin cubrir.
- Las condiciones compuestas de las reglas required|integer|exists fueron descompuestas en condiciones simples A (campo presente) y B (identificador válido y existente); cada una tomó verdadero y falso al menos una vez.
- Se identificaron y ejecutaron nueve caminos factibles: el exitoso, las salidas de error de la validación del request, las salidas de error de negocio observadas en la llamada al servicio y el camino con asignación cancelada.
- La caja negra confirmó el comportamiento esperado desde la perspectiva del usuario: clases de equivalencia, límites de observaciones, combinaciones de decisión y transiciones de estado.
- No se detectaron defectos funcionales en la función store durante esta ejecución. Las 49 pruebas finalizaron aprobadas.

7.2. Observaciones técnicas

- La instrucción when($exceptId, ...) de la actualización no forma parte de store; se menciona solo para dejar claro que no se le atribuye cobertura en este informe.
- Cuando hay varias reglas incumplidas, el sistema informa solo la primera; el orden es turno, micro, conductor, ruta, duplicidad de conductor y duplicidad de micro. Este comportamiento es consistente y predecible, pero debe conocerse para interpretar los mensajes.
- Las asignaciones canceladas se excluyen de la consulta de duplicidad. Esto permite reprogramar un conductor o un micro que tenía una asignación cancelada, sin borrar el historial.
- La creación no toma hora_salida ni hora_llegada del formulario; esos valores se gestionan en los flujos de inicio y finalización. La prueba confirma que ambos campos quedan nulos al crear.
- Las pruebas se ejecutaron sobre SQLite en memoria. Las restricciones de claves foráneas con comportamiento RESTRICT se validan en MySQL/MariaDB; se recomienda repetir la suite en ese motor antes de la puesta en producción.

7.3. Recomendaciones

- Mantener la suite de 49 casos como prueba de regresión de la función store y ejecutarla después de cada cambio en las validaciones o en el modelo.
- Agregar pruebas de interfaz que verifiquen la visualización de los mensajes de error junto a cada campo, complementando las pruebas HTTP actuales.
- Extender el mismo esquema de caja blanca y caja negra a los procedimientos iniciar(), finalizar() y cancelar(), que completan el ciclo de vida de la asignación.
- Probar el flujo en MySQL/MariaDB para verificar las claves foráneas, los enums y el comportamiento de las consultas de duplicidad en el motor de producción.
- Documentar en el manual de usuario el orden de validación y el significado de cada mensaje, para reducir la incertidumbre cuando hay más de un dato inválido.

Anexo A. Casos automatizados y entorno de ejecución

Los casos se implementaron en un archivo de prueba de Laravel. Cada método representa un caso del informe y utiliza aserciones sobre la respuesta HTTP, los errores de sesión y el estado de la base de datos. El archivo se ejecuta con el comando indicado en la sección 6.1.

Grupo	Métodos de prueba	Cantidad
Caja blanca · instrucciones	test_cb_i01 ... test_cb_i07	7
Caja blanca · decisiones	test_cb_d01 ... test_cb_d09	11
Caja blanca · condiciones	test_cb_c01 ... test_cb_c06	6
Caja blanca · caminos	test_cb_p01 ... test_cb_p08	8
Caja negra · equivalencia	test_cn_pe01 ... test_cn_pe03	3
Caja negra · valores límite	test_cn_vl01 ... test_cn_vl06	6
Caja negra · tabla de decisión	test_cn_td01 ... test_cn_td04	4
Caja negra · transición de estados	test_cn_te01 ... test_cn_te04	4

Anexo B. Diferencia entre caja blanca y caja negra

Aspecto	Caja blanca	Caja negra
¿Se examina el código?	Sí. Se analiza la estructura interna y la lógica.	No. Se analiza el comportamiento externo.
Enfoque	Instrucciones, decisiones, condiciones y caminos.	Requisitos, entradas, salidas y reglas de negocio.
Base para diseñar las pruebas	Código fuente de la función store.	Reglas de negocio de la asignación de turno.
Ejemplo en este informe	Comprobar las ramas de la validación del request y de la llamada al servicio dentro de store.	Verificar que un micro inactivo o inexistente sea rechazado.
Métodos aplicados	Instrucciones, decisiones, condiciones y caminos.	Equivalencia, valores límite, tabla de decisión y transición de estados.
Aporte principal	Detecta código no ejecutado y caminos lógicos no probados.	Confirma que el sistema cumple las reglas desde la perspectiva del usuario.

Conclusión: ambos enfoques son complementarios. La caja blanca permitió comprobar que la función store ejecuta sus instrucciones y recorre sus ramas y caminos; la caja negra permitió verificar que el comportamiento observable coincide con las reglas de negocio. La combinación de los dos enfoques proporciona una confianza mayor que cualquiera de ellos por separado.

Validación del documento

Rol	Nombre	Fecha	Firma
Elaborado por		5 de noviembre de 2026	
Revisado por			
Aprobado por			